

PyEtSimul

An Open-Source Python Framework for Eye-Tracking Simulation

`pypi` `v3.4.0` `downloads` `3k` `License` `GPL-3.0` `docs` `passing` `DOI` `10.1145/3806023`

Mohammadhossein Salari¹ · Diederick C. Niehorster² · Dan Witzner Hansen³ · Roman Bednarik¹



UNIVERSITY OF
EASTERN FINLAND



LUND
UNIVERSITY

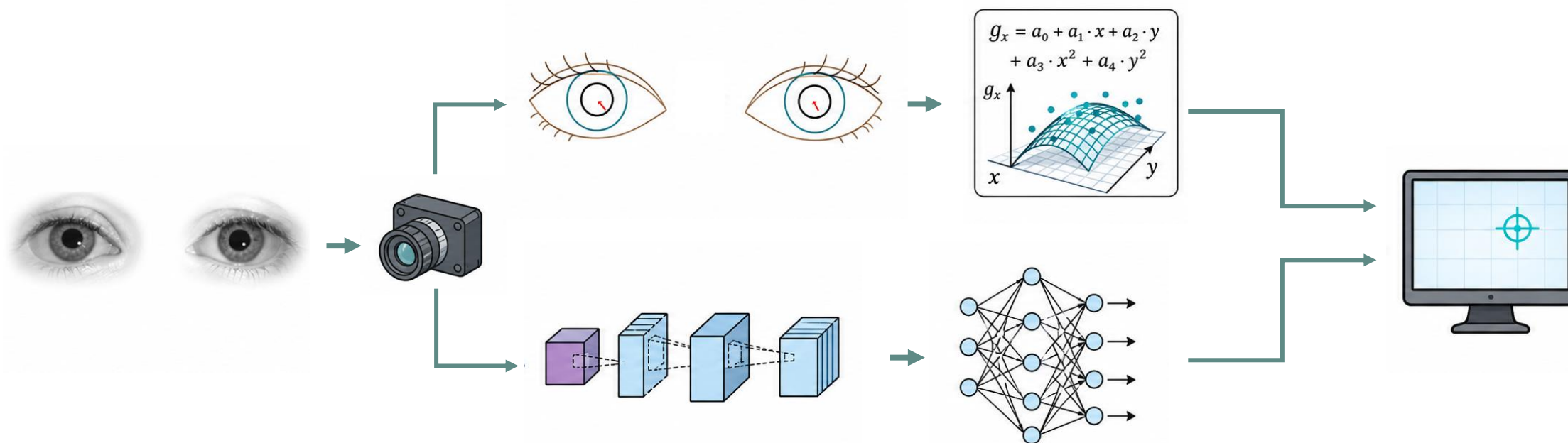
3 IT UNIVERSITY OF COPENHAGEN



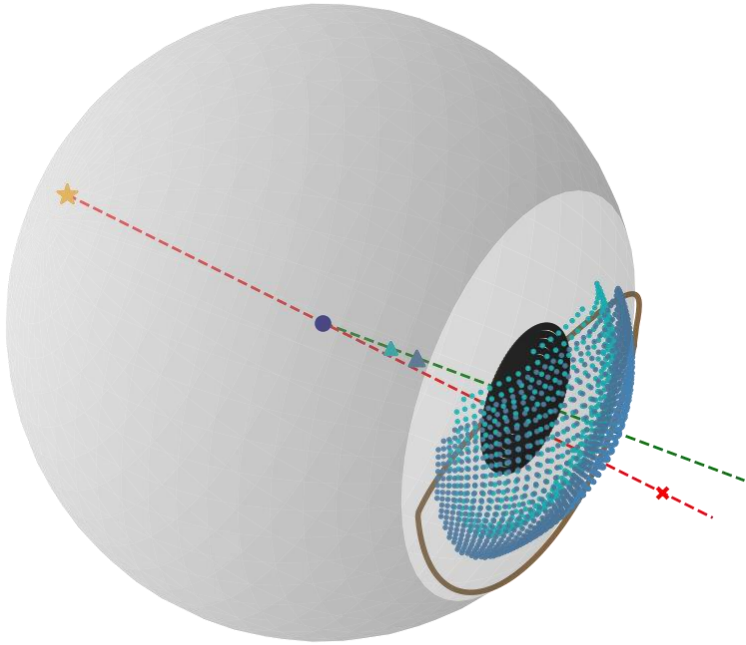
Funded by
the European Union

Funded by the EU Horizon Europe programme, grant number 101072410 Eyes4ICU project.

Video-based eye tracking: from eye images to gaze data



PyEtSimul: a feature-based simulator

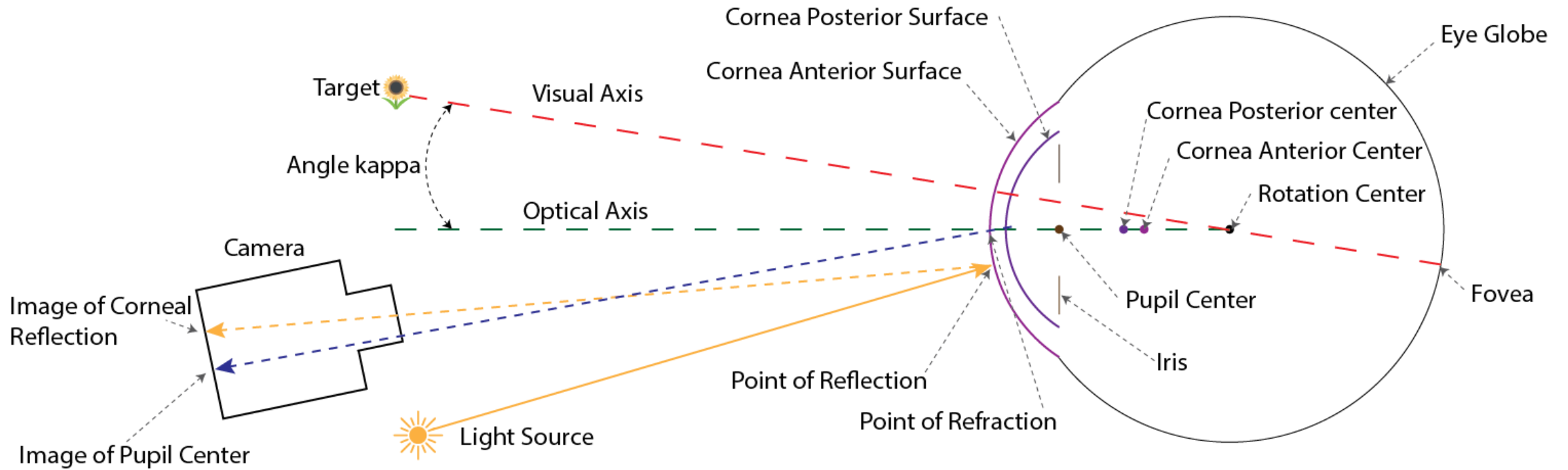


Models the eye's geometry explicitly at the feature level — exact pupil and corneal-reflection positions

Every eye and setup parameter can be set

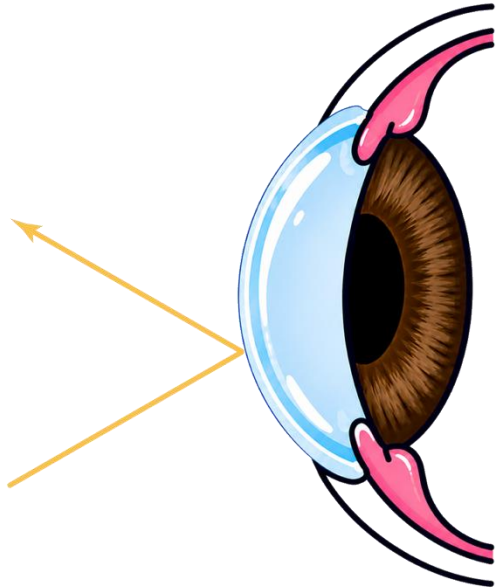
It opens the black box of eye tracking: isolate one factor, change it, and watch its effect

The eye model



Le Grand two-sphere eye model (Le Grand 1957; Le Grand & El Hage 2013)

Corneal reflection



Corneal reflection (glint)

$$\text{Normal: } \mathbf{n}(a) = \frac{a \mathbf{d}_c + (1 - a) \mathbf{d}_l}{\|a \mathbf{d}_c + (1 - a) \mathbf{d}_l\|}$$

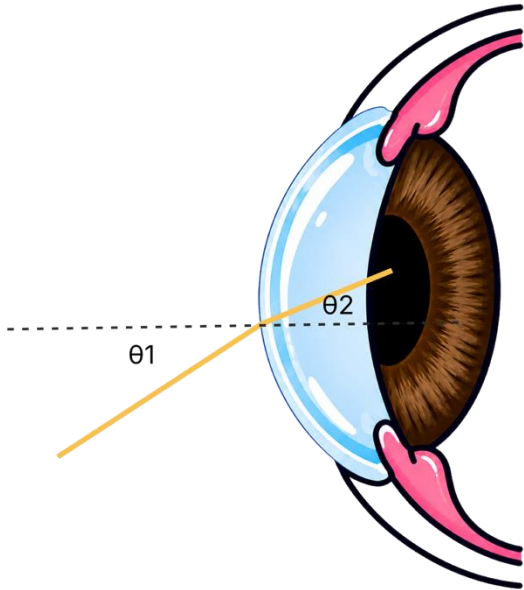
$$\text{Root function: } f(a) = \arccos(\mathbf{n}(a) \cdot \mathbf{u}_c) - \arccos(\mathbf{n}(a) \cdot \mathbf{u}_l) = 0$$

Solved with Brent's method

$\mathbf{d}_c, \mathbf{d}_l$ = directions to camera/light

$\mathbf{u}_c, \mathbf{u}_l$ = unit vectors to camera/light

Corneal refraction



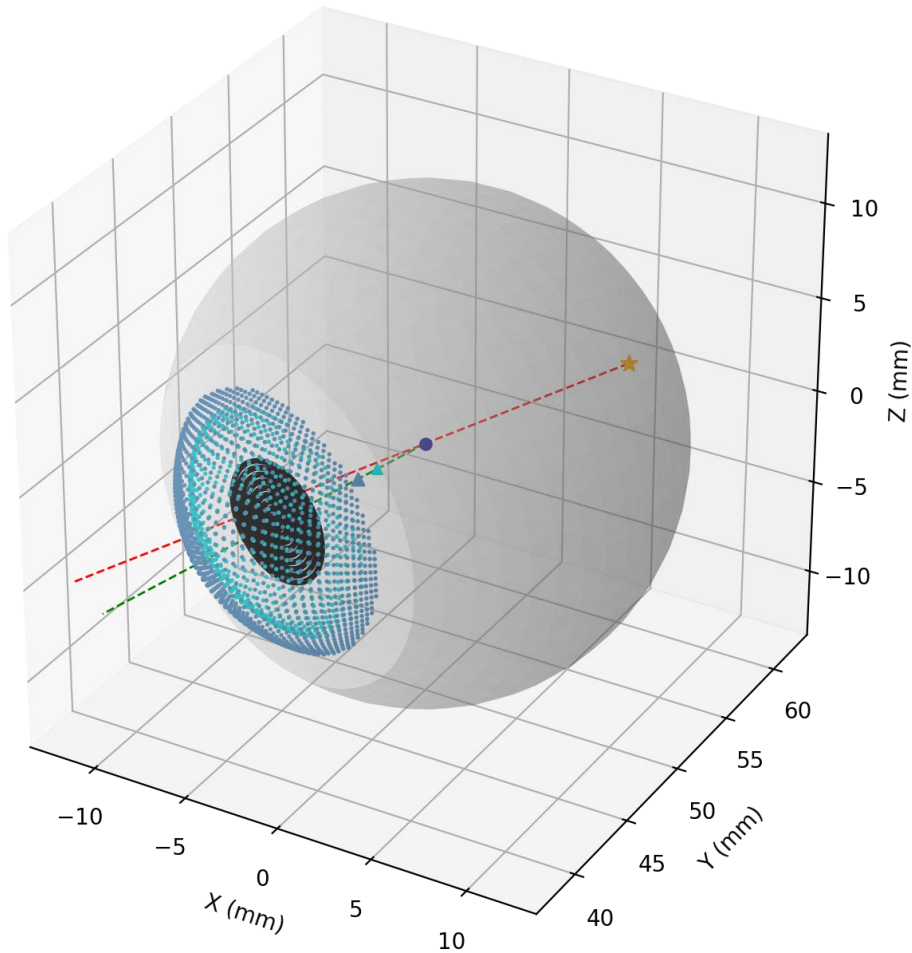
Corneal refraction (pupil)

$$\text{Snell: } n_1 \sin \theta_1 = n_2 \sin \theta_2$$

$$\text{Root function: } f(a) = n_{\text{air}} \sin \theta_{\text{camera}} - n_{\text{cornea}} \sin \theta_{\text{pupil}} = 0$$

Solved numerically

Conic cornea



Conic cornea

Surface: $(x - c_x)^2 + (y - c_y)^2 + (1 + k)(z - c_z)^2 - 2R(z - c_z) = 0$

Apex at $z = -\frac{R}{1 + k}$; $k < 0$ prolate, $k = 0$ sphere, $k > 0$ oblate

Normal: $\mathbf{n} = \nabla F = (2(x - c_x), 2(y - c_y), 2(1 + k)(z - c_z) - 2R)$

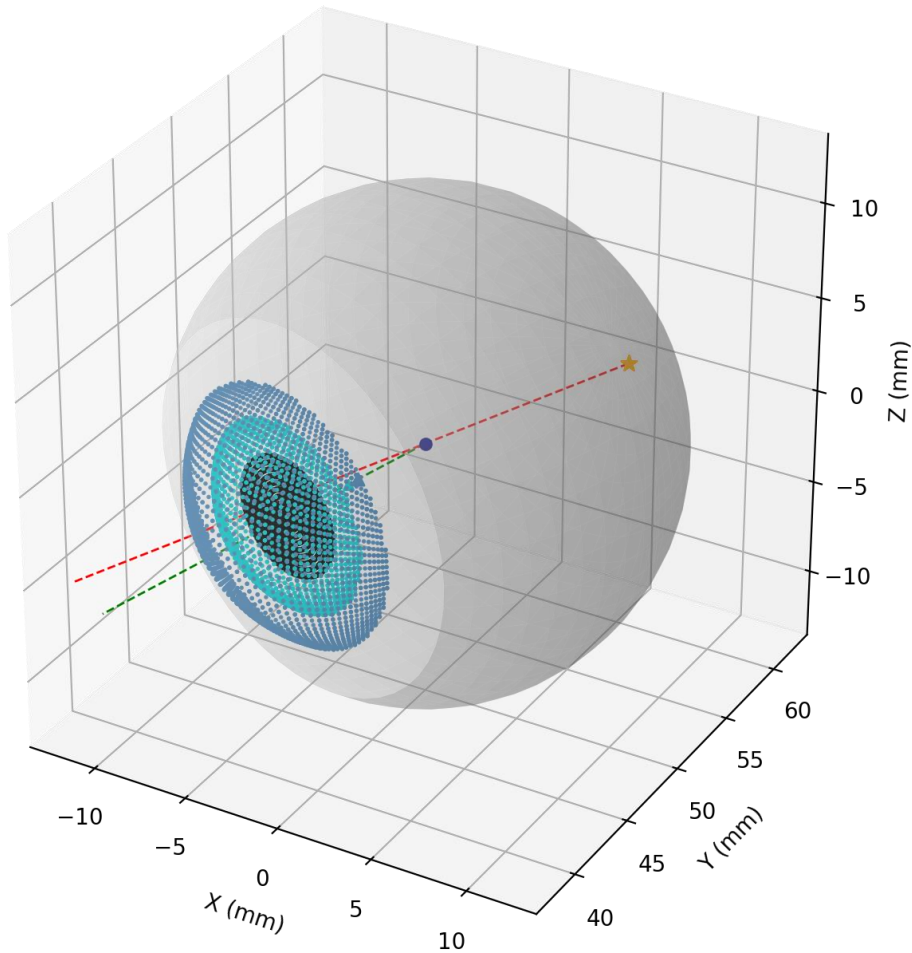
Fovea: $\mathbf{f} = \rho (\sin \alpha \cos \beta, \sin \beta, \cos \alpha \cos \beta)$, $\rho = L/2$

Angle κ : $\kappa = \arccos|\hat{\mathbf{v}} \cdot \hat{\mathbf{o}}|$, $\hat{\mathbf{o}} = (0, 0, -1)$

R = apex radius; k = conic constant; $\mathbf{c}$ = center; L = axial length

α, β = fovea displacement angles; $\hat{\mathbf{v}}$ = visual axis; $\hat{\mathbf{o}}$ = optical axis

Spherical cornea



Spherical cornea ($k = 0$ special case of the conic)

$$\text{Surface: } (x - c_x)^2 + (y - c_y)^2 + (z - c_z)^2 = R^2 \quad \Longleftrightarrow \quad \|\mathbf{p} - \mathbf{c}\| = R$$

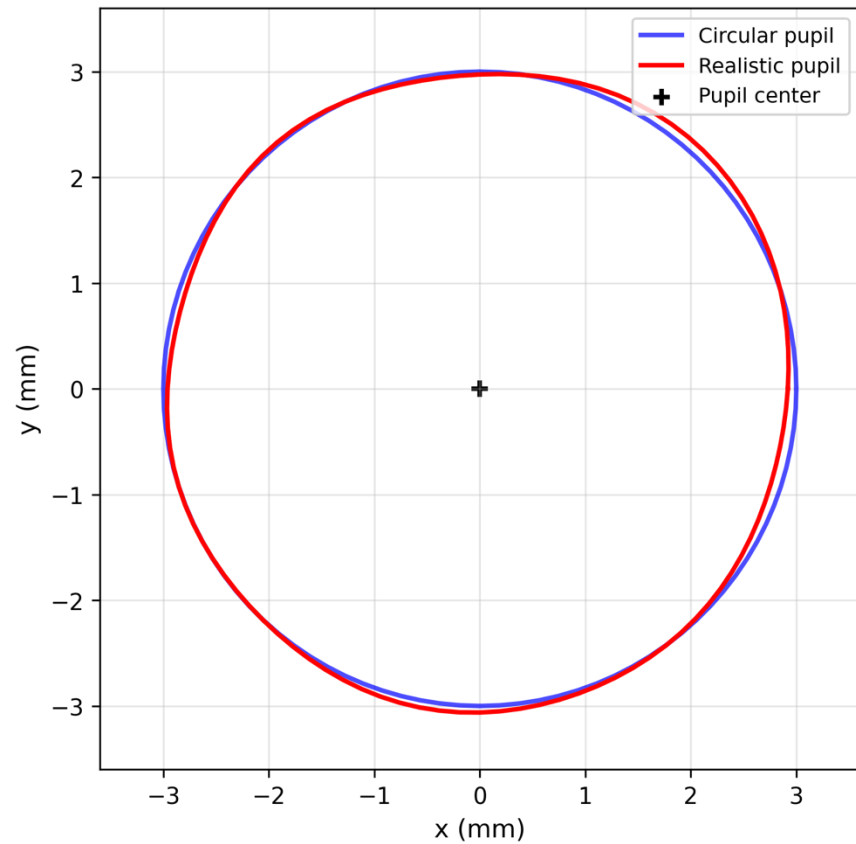
$$\text{Normal: } \mathbf{n} = \frac{\mathbf{p} - \mathbf{c}}{\|\mathbf{p} - \mathbf{c}\|}$$

$$\text{Apex: } \mathbf{c} + (0, 0, -R)$$

$$\text{Scaling (Boff \& Lincoln 1988): } s = \frac{R}{R_{\text{ref}}}$$

R = corneal radius; $\mathbf{c}$ = center; $\mathbf{p}$ = surface point; s = scale factor

Pupil model



Pupil model

Elliptical: $\mathbf{p}(\alpha) = \mathbf{c} + \cos \alpha \hat{\mathbf{x}} + \sin \alpha \hat{\mathbf{y}}, \quad \alpha \in [0, 2\pi)$

Realistic (Wyatt 1995): $R(\theta) = \bar{r} + \sum_{n=2}^N r_n \cos(n(\theta - \varphi_n))$

Noncircularity: $\text{NC}^2 = \frac{1}{2} \sum_{n=2}^N \left(\frac{r_n}{\bar{r}} \right)^2$

Decentration (Wildenmann 2013): $\Delta x = c_x (d - d_0), \quad \Delta y = c_y (d - d_0)$

$\mathbf{c}$ = center; $\hat{\mathbf{x}}, \hat{\mathbf{y}}$ = semi-axis vectors; $\bar{r}$ = mean radius

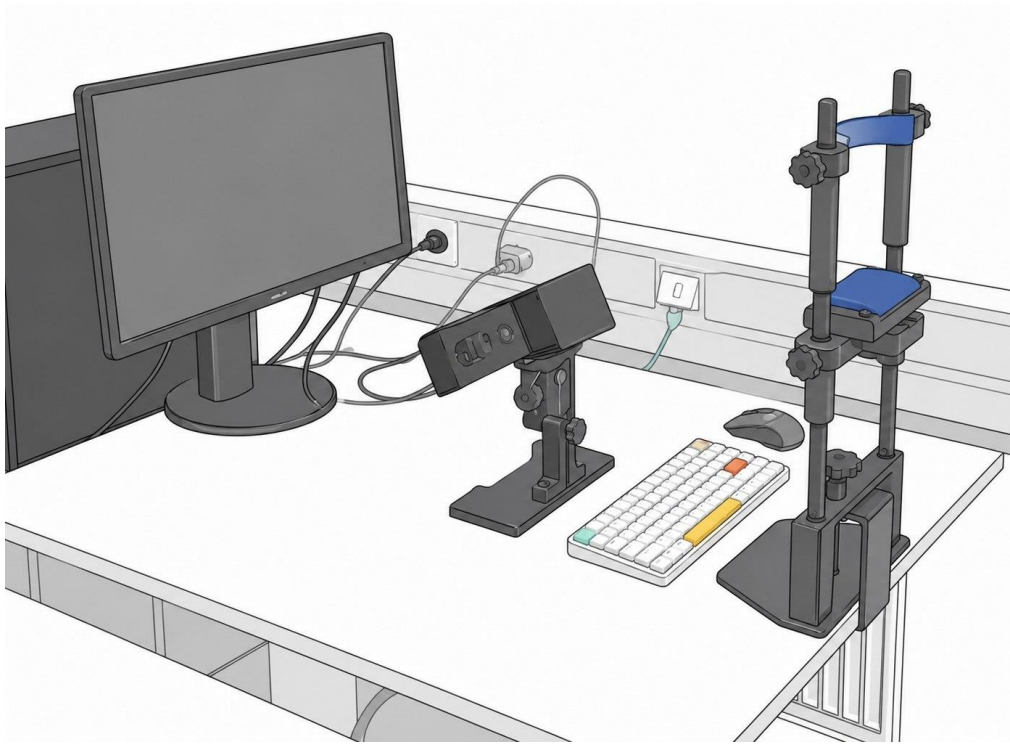
r_n, φ_n = harmonic amplitude/phase; d = diameter; d_0 = baseline; c_x, c_y = coeffs (mm/mm)

PyEtSimul: built on et_simul (Böhme et al. 2008), substantially extended

	et_simul	PyEtSimul
Spherical cornea	✓	✓
Pinhole camera	✓	✓
Pupil + corneal-reflection projection	✓	✓
Conic cornea	✗	✓
Non-circular pupil + decentration	✗	✓
Eyelid occlusion	✗	✓
Camera lens distortion + noise	✗	✓
Multiple cameras + import real calibration	✗	✓
Systematic parameter sweeps + evaluation	✗	✓
Built-in gaze estimation algorithms*	✓	✓
License	 MATLAB (paid)	 Python (free)

* both have gaze estimation algorithms, but different models — PyEtSimul ships 7 polynomial models + homography + easy registration of custom models

Setup for the demo



Our EyeLink 1000 Plus setup at UEF / Lund

- *eye-to-screen ~83.5 cm*
- *camera-to-eye ~47.6 cm*
- *IR light ~19.5 cm beside the camera*

Stampe (1993) — cross-term-free 2nd-order polynomial + per-quadrant corner correction

① Polynomial (inner 5 pts, no xy)

$$\tilde{x} = a_0 + a_1x + a_2y + a_3x^2 + a_4y^2$$
$$\tilde{y} = b_0 + b_1x + b_2y + b_3x^2 + b_4y^2$$

② Corner correction (outer 4 corners → restores xy)

$$\text{gaze}_x = \tilde{x} + k_Q^x (\tilde{x} - \bar{x})(\tilde{y} - \bar{y})$$
$$\text{gaze}_y = \tilde{y} + k_Q^y (\tilde{x} - \bar{x})(\tilde{y} - \bar{y})$$

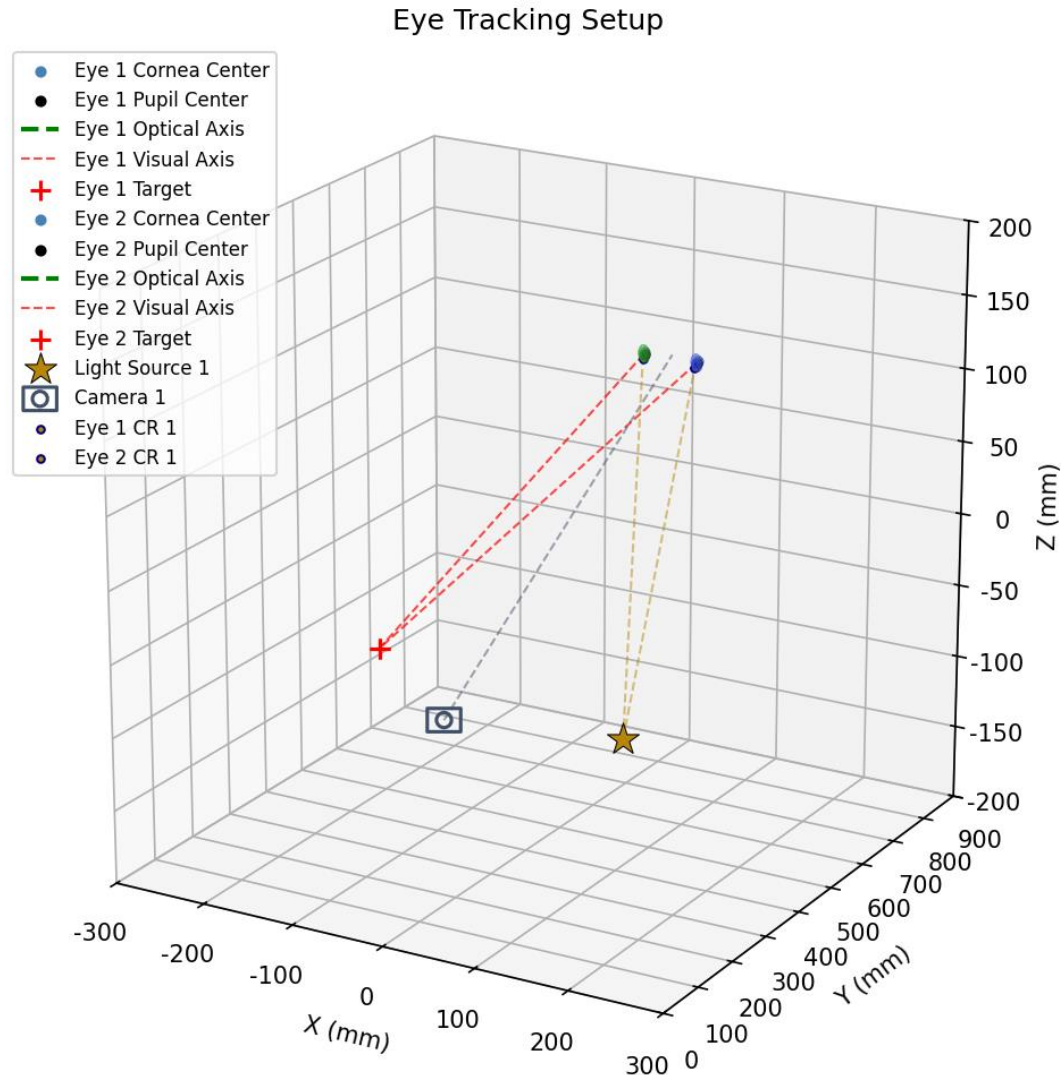
(x, y) = pupil – corneal reflection

$(\bar{x}, \bar{y})$ = target centroid

Q = quadrant

Demo: building a Stampe (1993) eye tracker

① Define a setup, in a few lines of Python

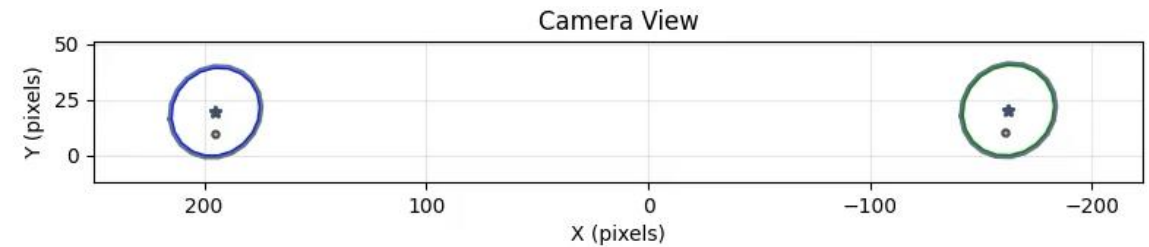
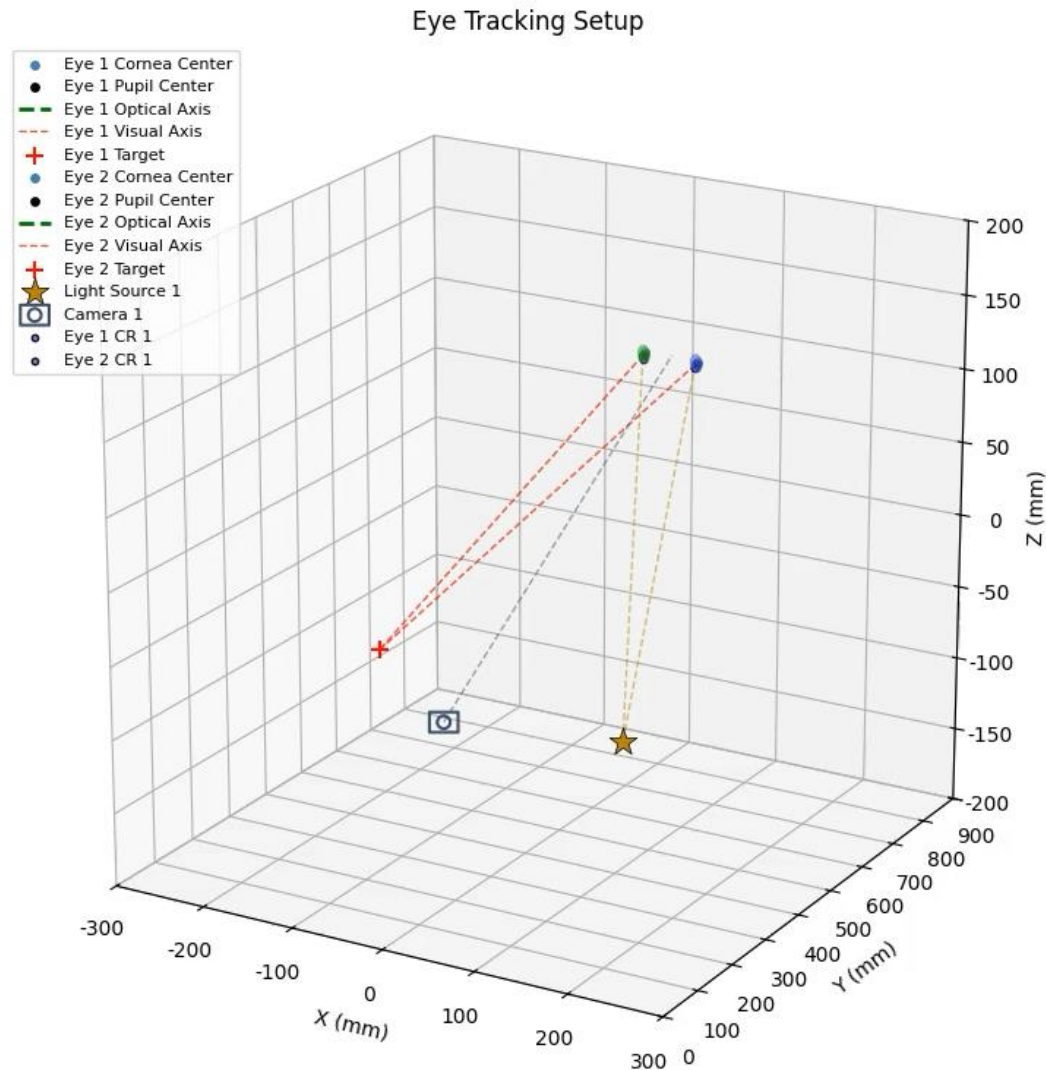


And an infrared light source to make a glint (corneal reflection):

```
1 from pyetsimul.core import Camera, Eye, Light
2 from pyetsimul.types import Position3D
3
4 right_eye = Eye()
5 right_eye.position = Position3D(30, 835, 85)
6 left_eye = Eye()
7 left_eye.position = Position3D(-30, 835, 85)
8
9 camera = Camera()
10 camera.position = Position3D(-100, 420, -125)
11 camera.point_at_binocular(left_eye.position,
12                           right_eye.position)
13
14 light = Light(position=Position3D(95, 435, -115))
```

Demo: building a Stampe (1993) eye tracker

② It computes the pupil + glint for any gaze

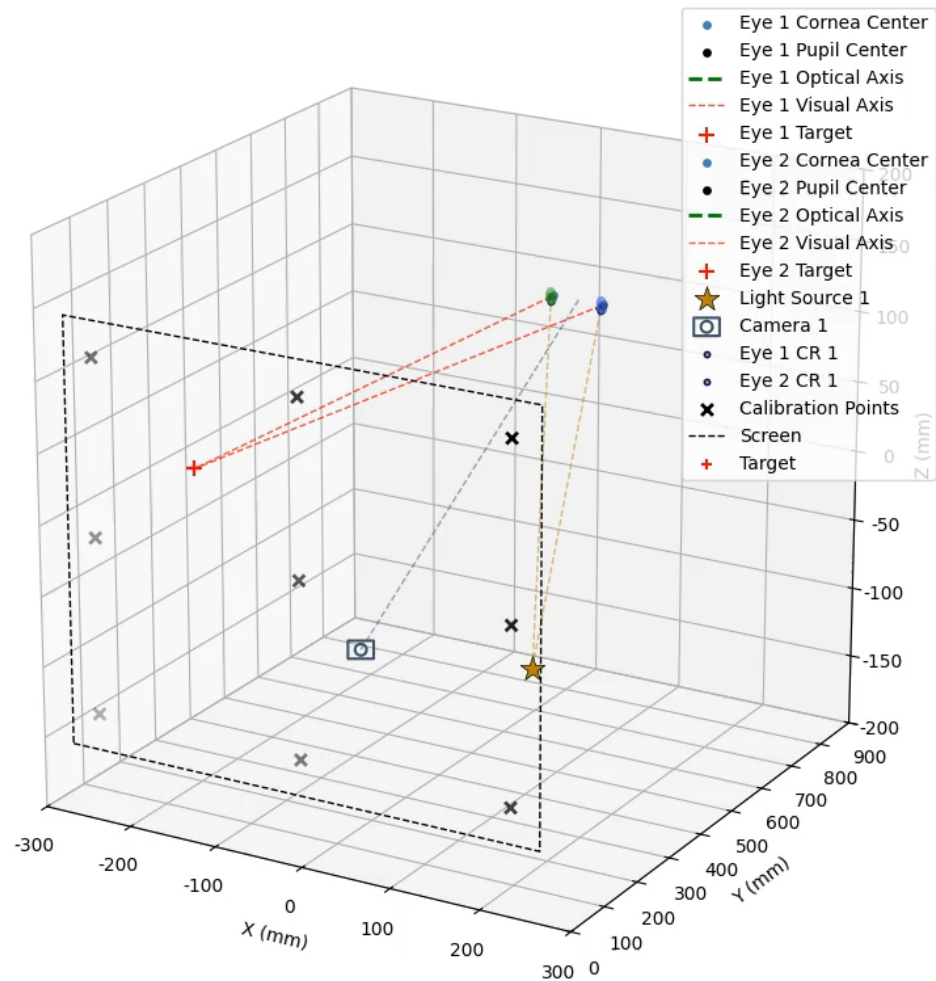


```
16 target = Position3D(0, 0, 0)
17 for eye in (left_eye, right_eye):
18     eye.look_at(target)
19     image = camera.take_image(eye, [light])
20     image.pupil_center          # pupil centre    (px)
21     image.corneal_reflections  # glint (CR)     (px)
22     image.pupil_boundary       # pupil outline  (px)
```

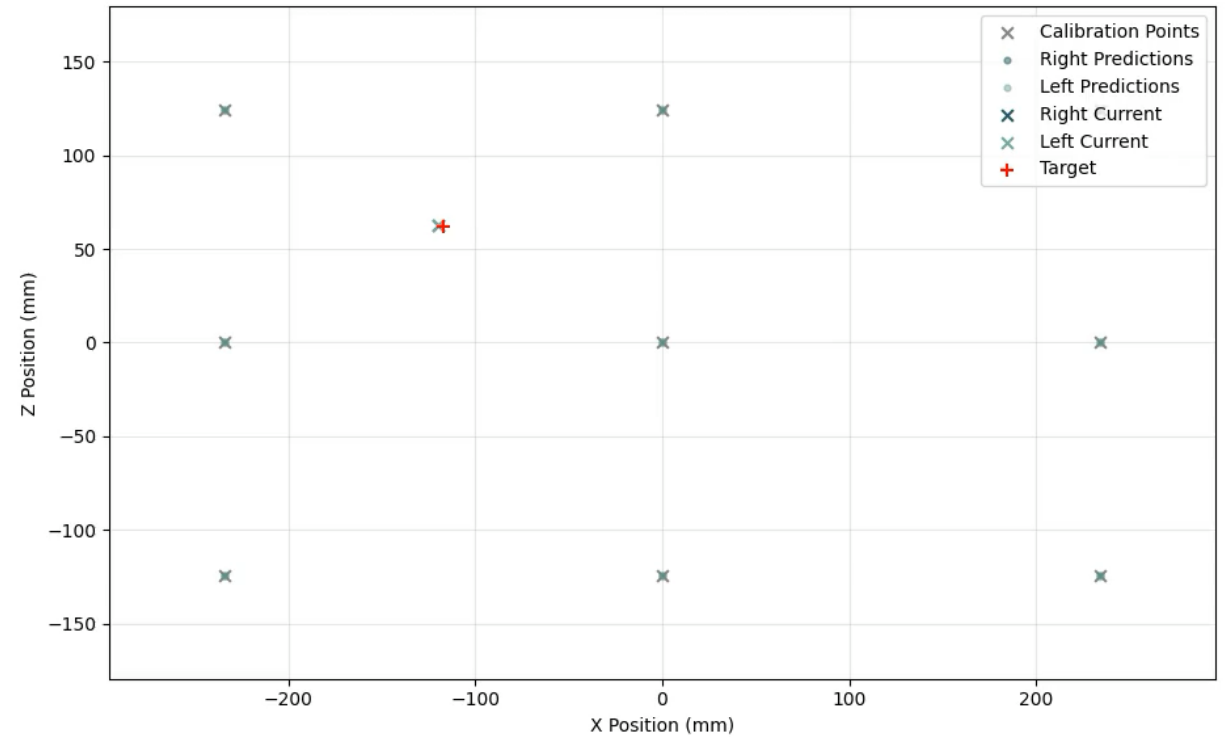
Demo: building a Stampe (1993) eye tracker

③ Calibrate → a working eye tracker

Eye Tracking Setup



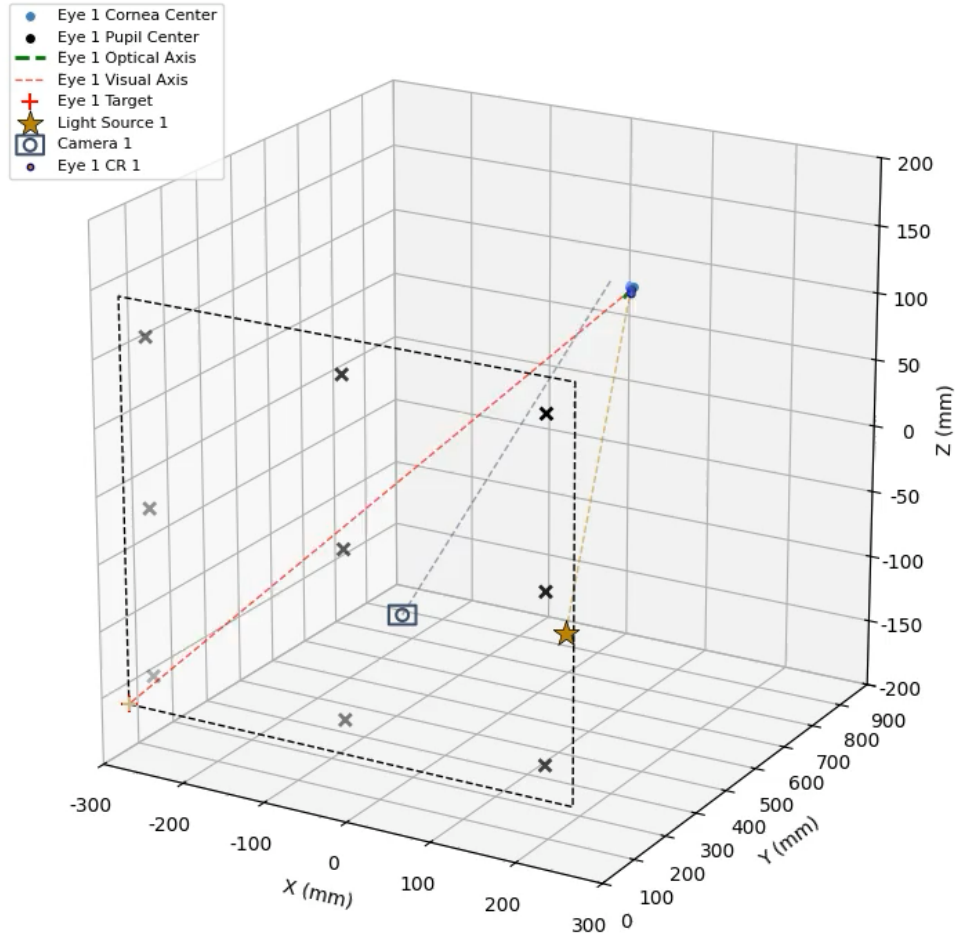
Calibration Analysis
Current gaze error: 0.1988° (2.96mm) avg across 2 eyes
Calibration errors — Avg: 0.000° | Max: 0.000°



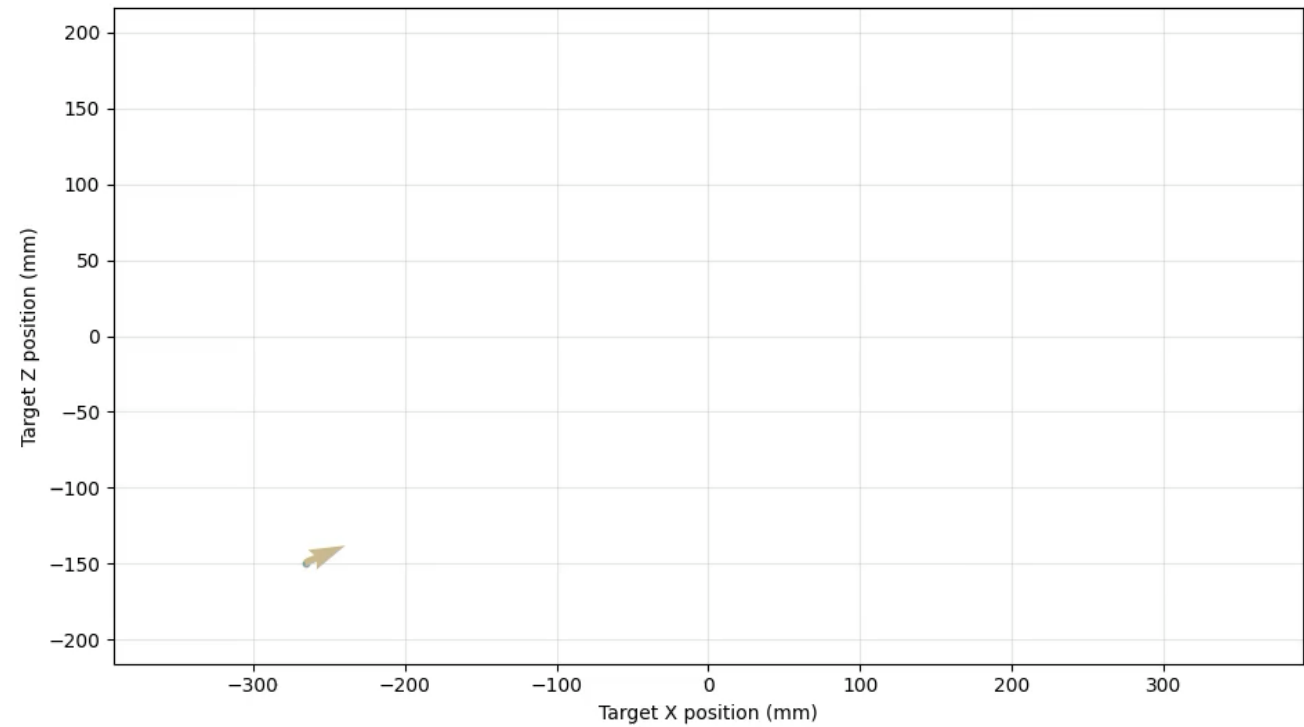
Demo: building a Stampe (1993) eye tracker

Screen Test — the gaze moves across the screen

Screen test — gaze sweeps the screen



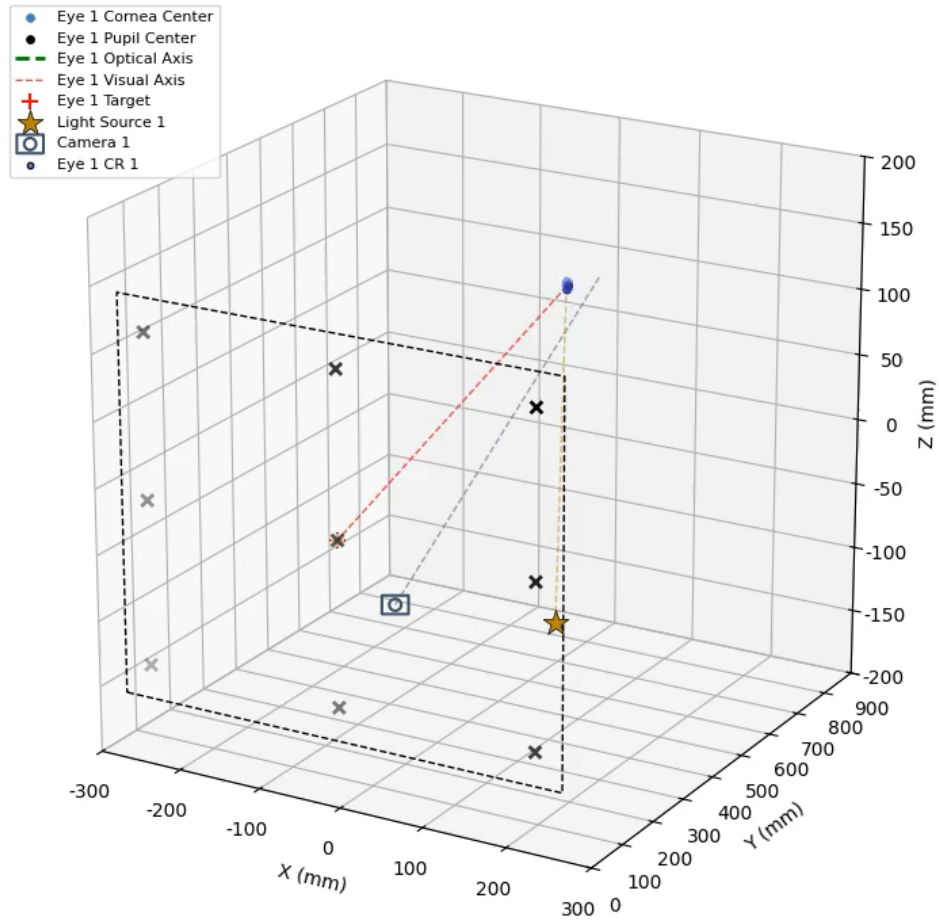
Gaze error across the screen
Points: 1/100 Mean error: 0.16° (arrows ×10)



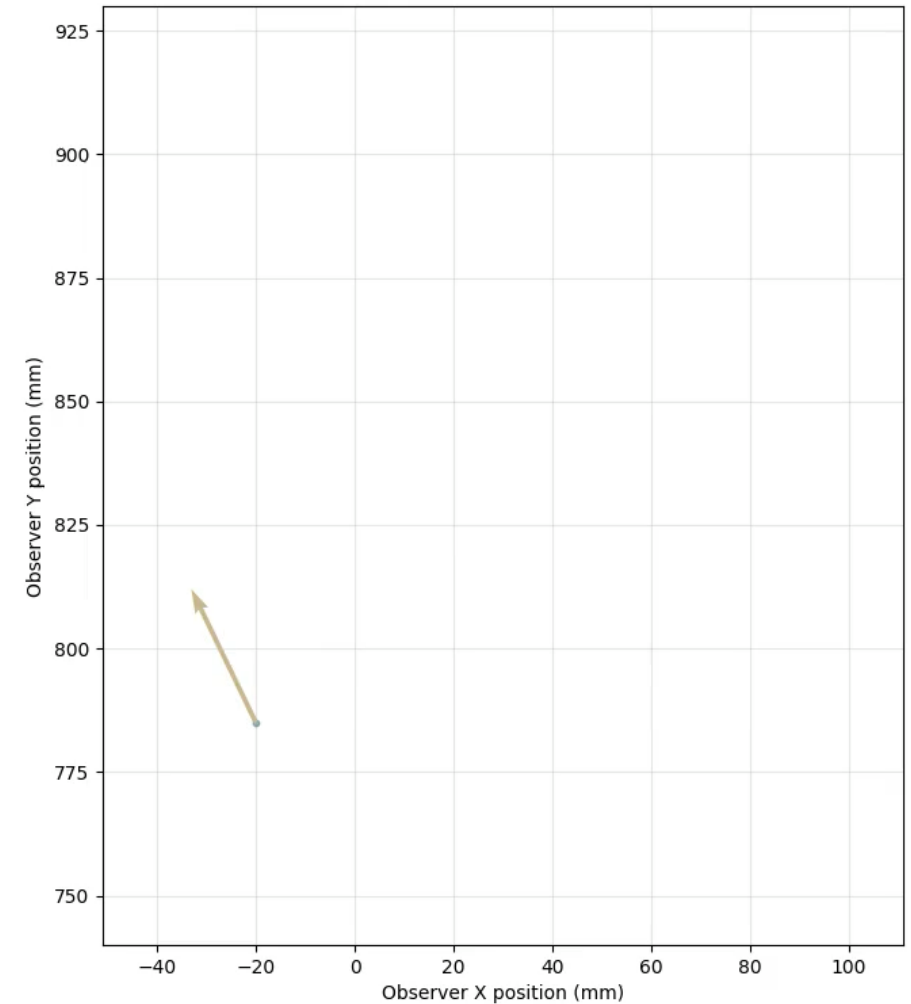
Demo: building a Stampe (1993) eye tracker

Observer Test — the eye position moves

Observer test — head position moves

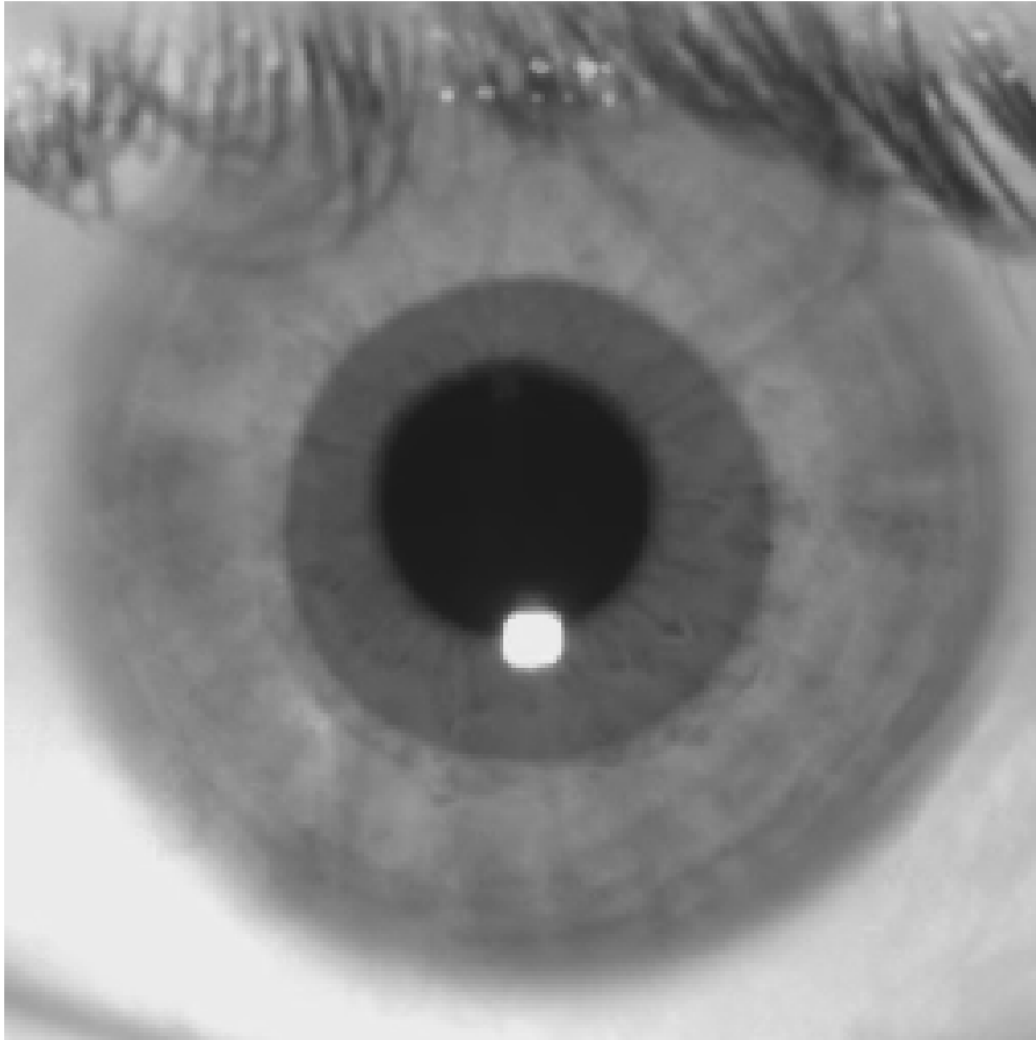


Gaze error vs. eye position
Points: 1/100 Mean error: 7.34°



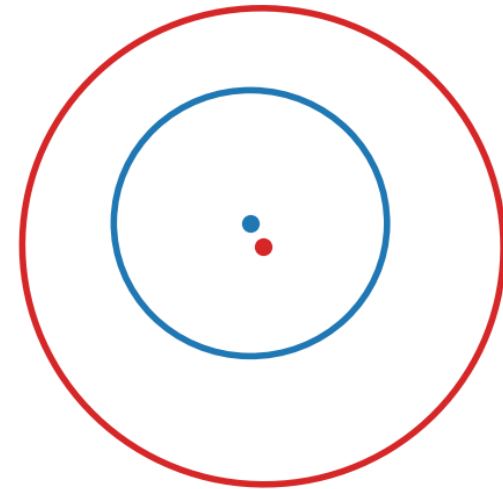
The pupil-size artifact (PSA)

Eyes overlay (glint-aligned)



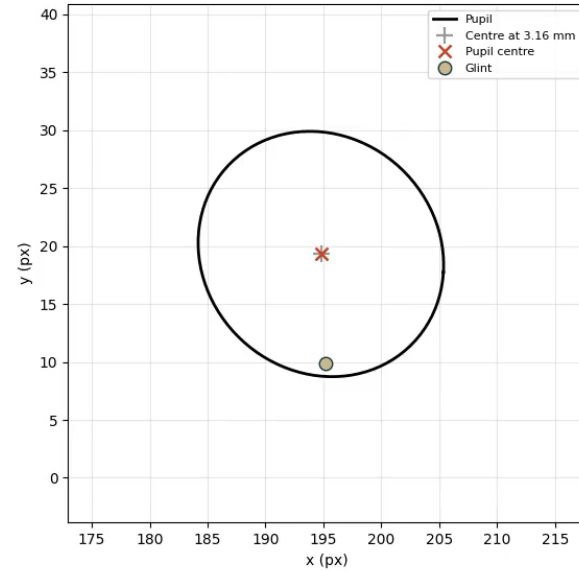
Pupil-centre shift = 0.31 mm (6.4 px)

- Constricted pupil
- Dilated pupil
- Constricted pupil center
- Dilated pupil center

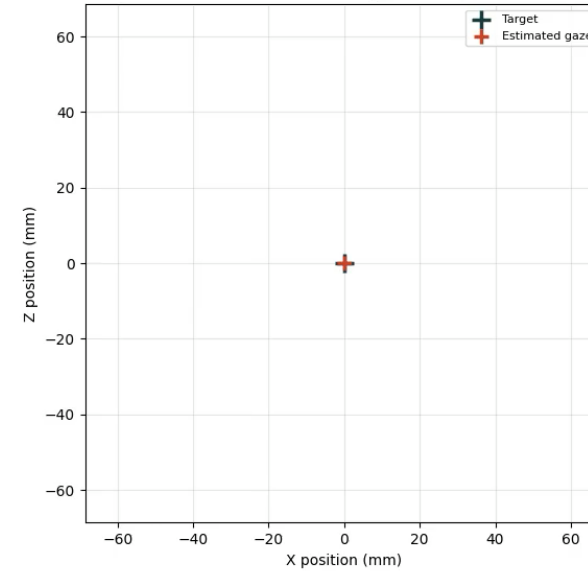


Only in simulation: the pupil-size artifact

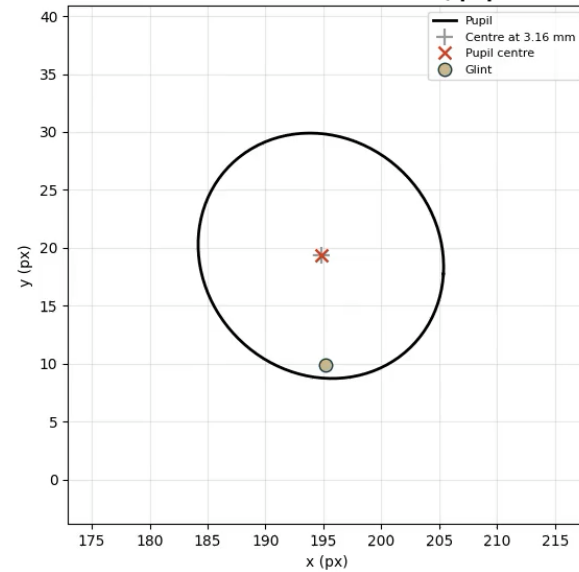
With decentration — camera view, pupil 3.2 mm



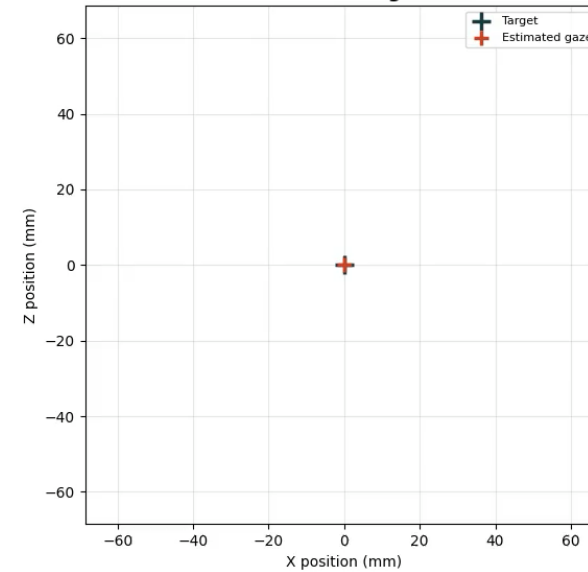
With decentration — gaze error 0.00°



Without decentration — camera view, pupil 3.2 mm



Without decentration — gaze error 0.00°



PyEtSimul vs other simulators

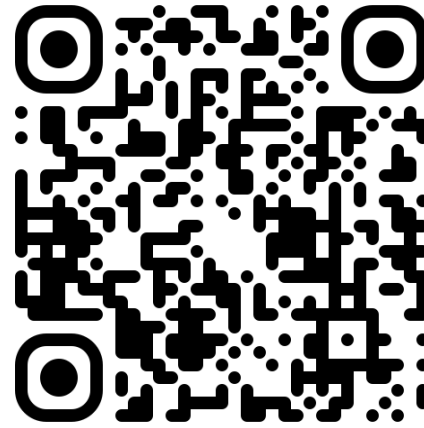
How PyEtSimul compares

- Complementary to image-rendering simulators (UnityEyes, NVGaze) that generate eye images to train deep-learning gaze models
- Feature-level access opens the black box — letting you isolate the factors that affect gaze
- The only open-source Python tool with a complete eye-tracking simulation workflow

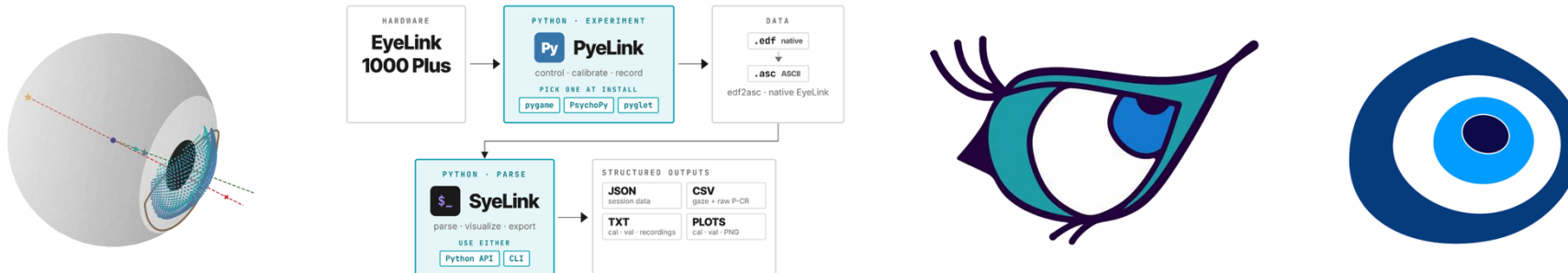
Limitations

- Feature-level / analytical — no image-level effects (detection errors, illumination, imaging artifacts)
- No limbus, iris texture, or crystalline lens
- Point light sources only (no spatial extent)
- No posterior corneal-surface refraction

Thank you! Any questions?



Get the code, slides, and more at: mh-salari.ir/news/ETRA2026



• Pupil

- Simple
- Starburst
- Swirski2D
- ExCuSe
- ElSe
- PuRe
- PuReST
- PupilLabs2D

• Glint:

- Simple

• Limbus

- Daugman integro-differential
- Daugman active contour
- Pupil-guided active contour

